INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

Department of Computer Science

LAB 3 HLD Template

System Modeling (UML/DFD) and UI Prototyping — Student Submission Template

Software Engineering and Project Management | Course Code: PCC-CS504 | Semester V
To be filled in during the lab session and submitted as Assignment 3

Team / Group No : Code Alpha – Group No (5)

Project Title: IEM Admission Management System

Team Members (Name — Roll No.) 1.: Subho Pramanick - 84

2.: Ranajit Chattaraj - 68

3.: Aratreeka Pahari - 80

4.: Swapna Swadhikta Behera - 79

Section / Batch: A - 2024–2028

Date: 30-08-2026

Faculty Signature:

How to Use This Worksheet

Just keep this sample next to your blank worksheet and go section by section. Start with the use cases to see who's using the system and what they do, then move to the class diagram to understand the data and relationships. The three-tier architecture shows you how to separate UI, logic, and database - that's the foundation of any good system. The decision table is especially helpful because it tells you who decides what and how. The key is consistency - don't call something "Student" in one diagram and "Applicant" in another. And remember, you're copying the structure, not the content - replace their library examples with your admission system's actual features. One section at a time, keep it simple, and you'll be fine.

0. YOUR PROBLEM STATEMENT

The IEM Admission System allows students to register, fill out their personal and academic information, upload required documents, and track their application status online. Administrators can verify documents, calculate merit based on academic performance and reservation policies, generate category-wise merit lists, and allot seats to eligible candidates. The system automates the entire admission workflow from application submission to final seat confirmation and receipt generation.

1. Use-Case Diagram

☐ All actors identified and labelled - Two actors are clearly identified and labelled: STUDENT and ADMIN

☐ System boundary drawn and labelled - The system boundary is drawn with "IEM ADMISSION SYSTEM" as the label, enclosing all use cases

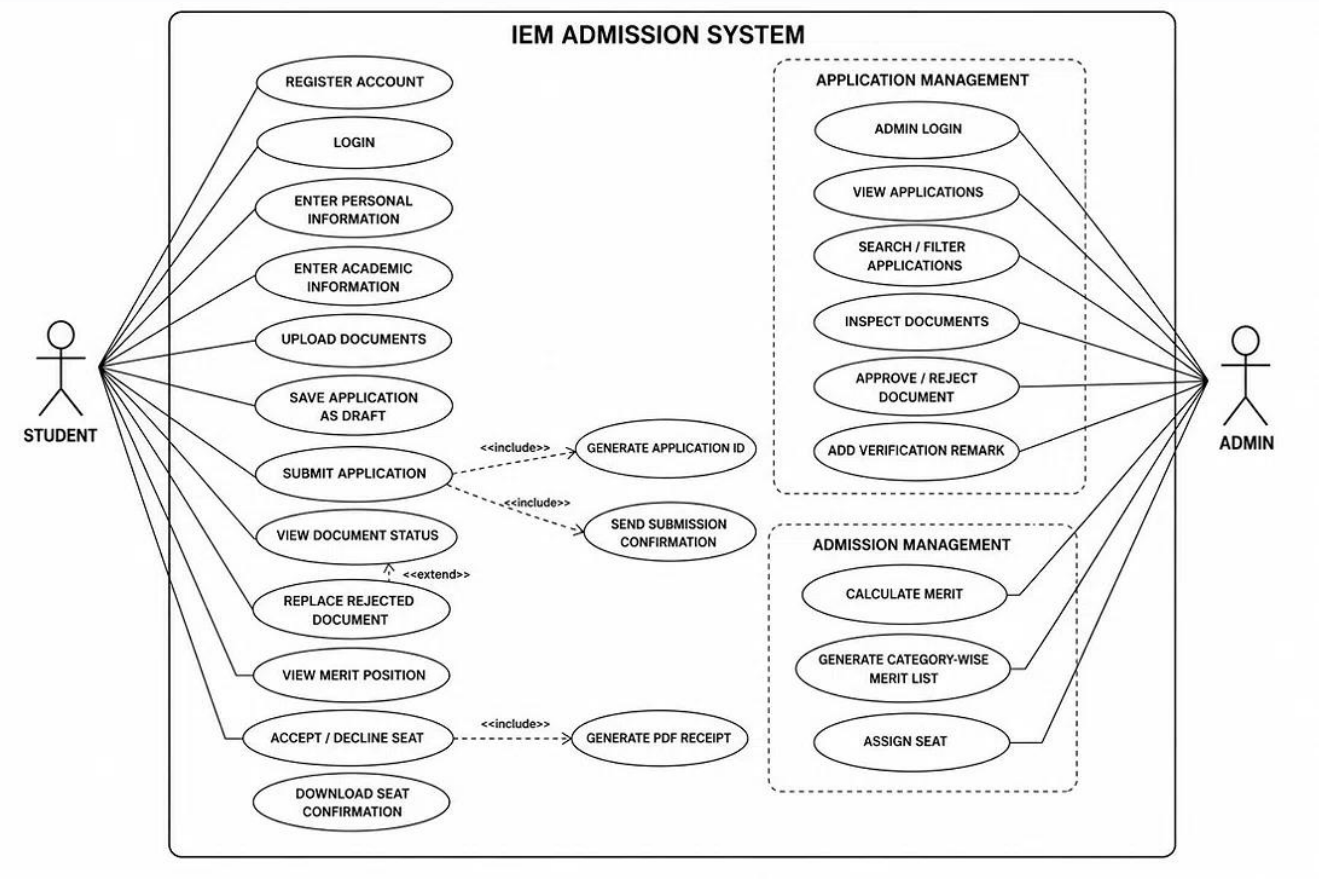
☐ Every use case is a distinct piece of functionality - All use cases represent specific actions:

STUDENT: REGISTER ACCOUNT, LOGIN, ENTER PERSONAL INFORMATION, ENTER ACADEMIC INFORMATION, UPLOAD DOCUMENTS, SAVE APPLICATION AS DRAFT, SUBMIT APPLICATION, VIEW DOCUMENT STATUS, REPLACE REJECTED DOCUMENT, VIEW MERIT POSITION, ACCEPT/DECLINE SEAT, DOWNLOAD SEAT CONFIRMATION

ADMIN: ADMIN LOGIN, VIEW APPLICATIONS, SEARCH/FILTER APPLICATIONS, INSPECT DOCUMENTS, APPROVE/REJECT DOCUMENT, ADD VERIFICATION REMARK, CALCULATE MERIT, GENERATE CATEGORY-WISE MERIT LIST, ASSIGN SEAT

☐ Associations connect the correct actor to the correct use case(s) - STUDENT is associated with all student-related use cases; ADMIN is associated with application management and admission management use cases

☐ Any «include» / «extend» relationship is labelled - REGISTER ACCOUNT includes LOGIN; no extend relationships are present in this diagram



2. DFD (Context + Level-1)

Required Elements Checklist:

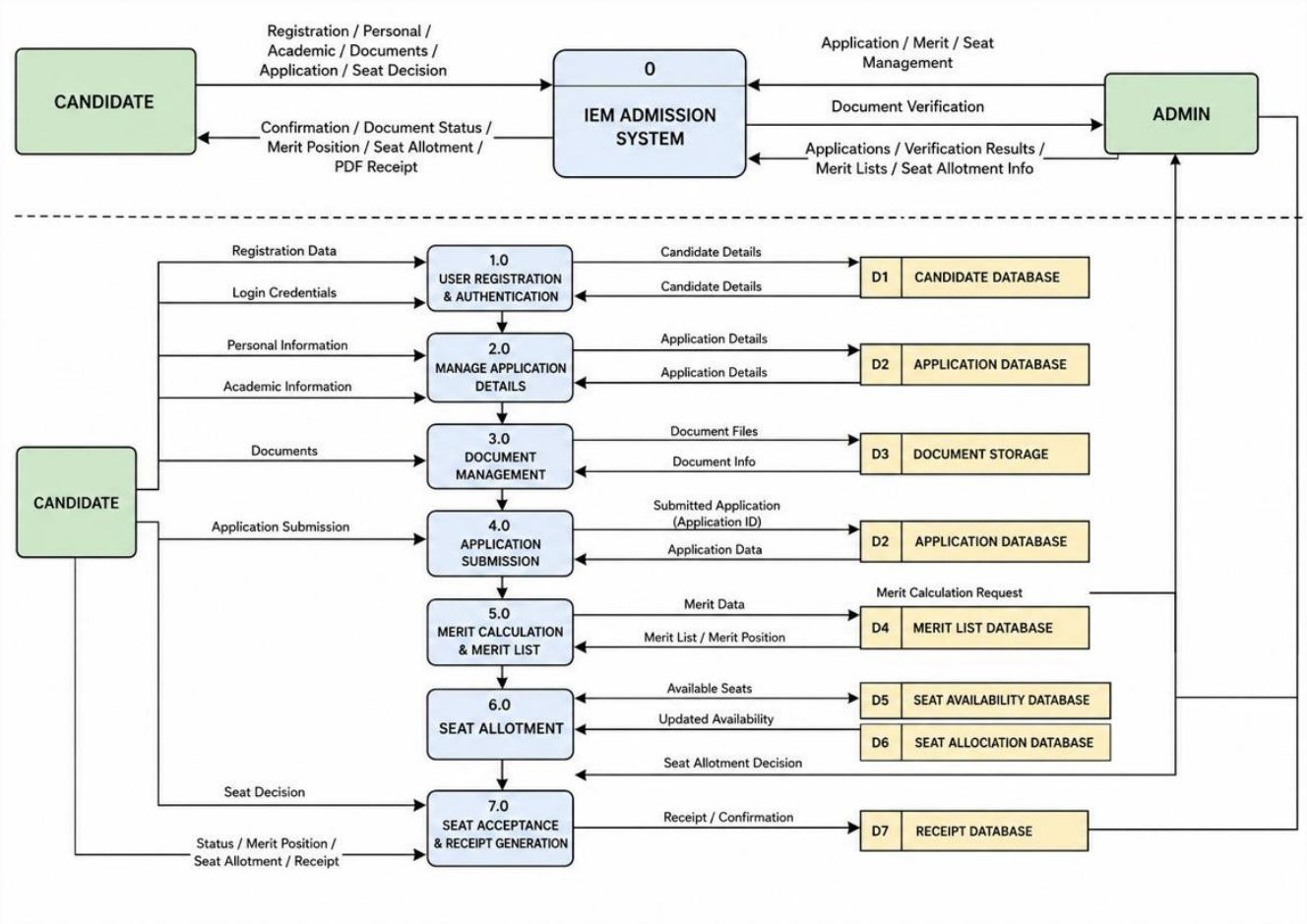
☐ All actors identified and labelled - CANDIDATE and ADMIN are clearly labelled as external entities

☐ System boundary drawn and labelled - "IEM ADMISSION SYSTEM" boundary encloses all processes

☐ Every process is a distinct piece of functionality - Six numbered processes: USER REGISTRATION & AUTHENTICATION (1.0), DOCUMENT MANAGEMENT (2.0), APPLICATION SUBMISSION (3.0), MERIT CALCULATION & MERIT LIST (4.0), SEAT ALLOTMENT (5.0), SEAT ACCEPTANCE & RECEIPT GENERATION (6.0)

☐ Data flows connect the correct entities to the correct processes - CANDIDATE sends registration data, documents, and application details; receives status updates, merit position, seat allotment, and receipt. ADMIN manages merit calculation, seat allotment, and seat acceptance processes

☐ Data stores are correctly identified and connected - CANDIDATE DATABASE, APPLICATION DATABASE, DOCUMENT STORAGE, and DOCUMENT DATABASE are present with proper connections to relevant processes



3. Activity Diagram

Required Elements Checklist:

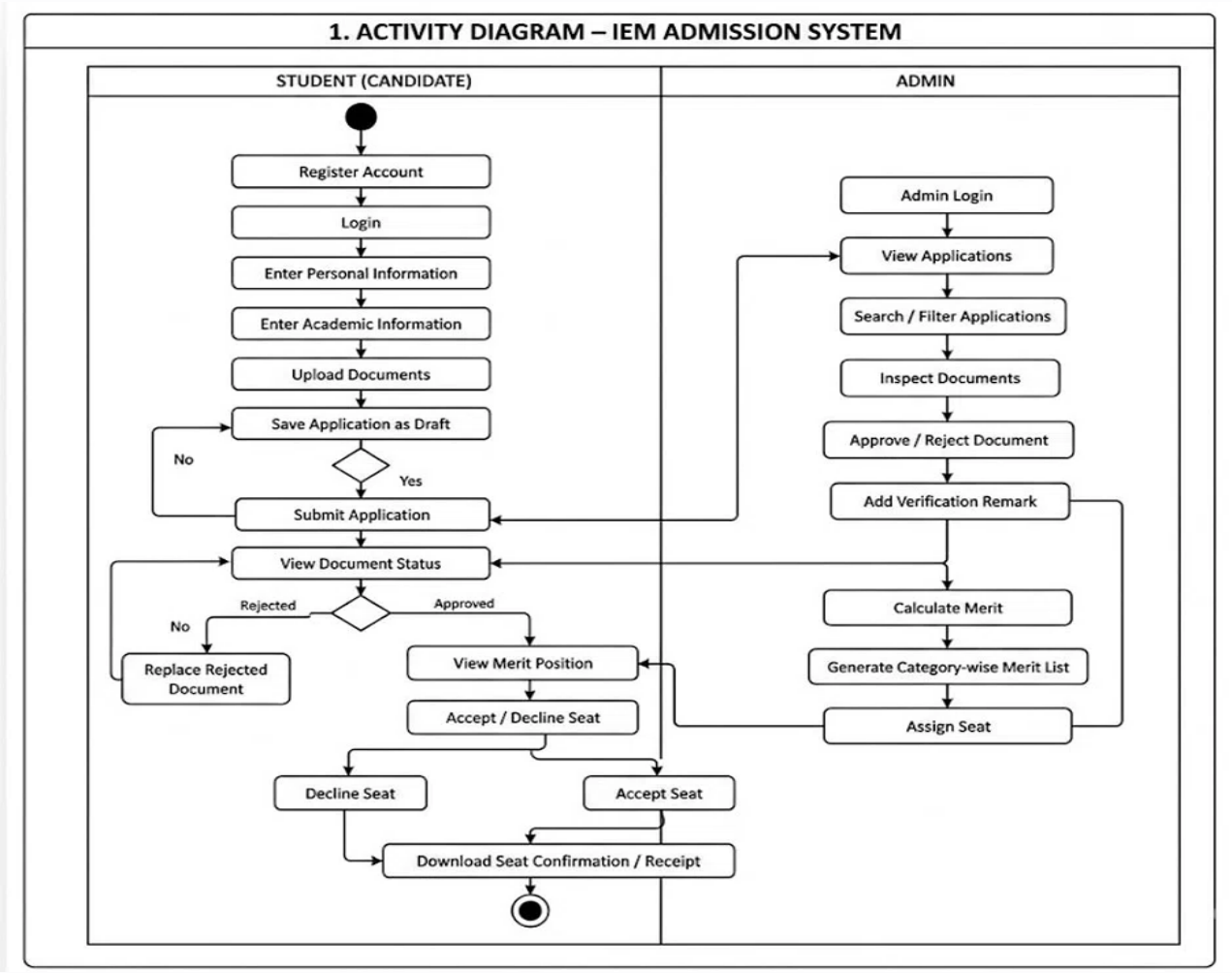
☐ Initial node present - Activity begins with "Register Account"

☐ Final node present - Activity ends with "Download Seat Confirmation / Receipt"

☐ All actions clearly named - Register Account, Login, Enter Personal Information, Enter Academic Information, Upload Documents, Save Application as Draft, Submit Application, View Document Status, Replace Rejected Document, Accept/Decline Seat, Download Seat Confirmation/Receipt

☐ Decision nodes present and labelled - Decision at "Save Application as Draft?" with branches going back to draft or proceeding to submit; Decision at "Document Status - Rejected" leading to Replace Rejected Document; Decision at "Accept/Decline Seat" with Accept leading to Download Receipt and Decline ending the process

☐ Swimlanes present - Two swimlanes clearly separate STUDENT (CANDIDATE) activities on one side and ADMIN activities on the other side



4. Sequence Diagram

Required Elements Checklist:

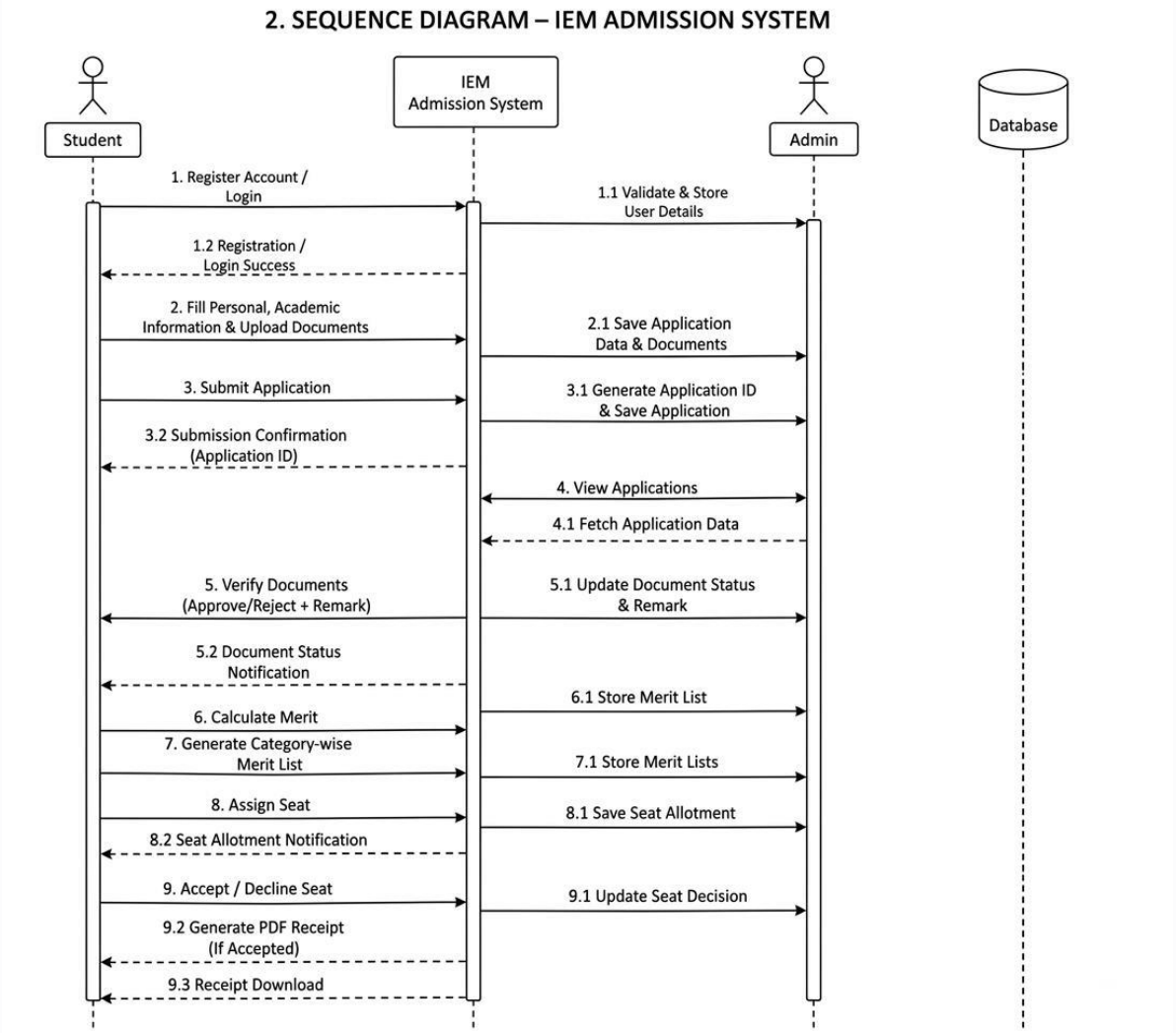
☐ All actors identified and labelled - Three lifelines: Student, IEM Admission System, and Admin

☐ Lifelines with dashed lines - Each actor has a dashed timeline extending downward

☐ Messages with arrows - All interactions shown with arrows between lifelines: Student registers/logs in, gets success response; Student fills info and uploads docs; Student submits application, gets confirmation with ID; Admin views and fetches applications; Admin verifies documents with remarks, student gets notification; Admin calculates merit, generates merit lists, assigns seat; student gets allotment notification; Student accepts/declines seat; system generates receipt if accepted and allows download

☐ Messages in sequential order - Numbered flow from 1.0 to 9.3 shows clear chronological sequence

☐ Return messages where appropriate - Responses like "Registration/Login Success", "Submission Confirmation", "Document Status Notification", "Seat Allotment Notification", and "Generate PDF Receipt" show system responding back to student



5. Design Diagram — UML Class Diagram / Structure Chart

Required Elements Checklist:

☐ All classes identified and named - Classes include User, Admin, Application, PersonalInfo, AcademicInfo, Document, DocumentVerification, Merit, MeritList, Seat, Payment, and Receipt

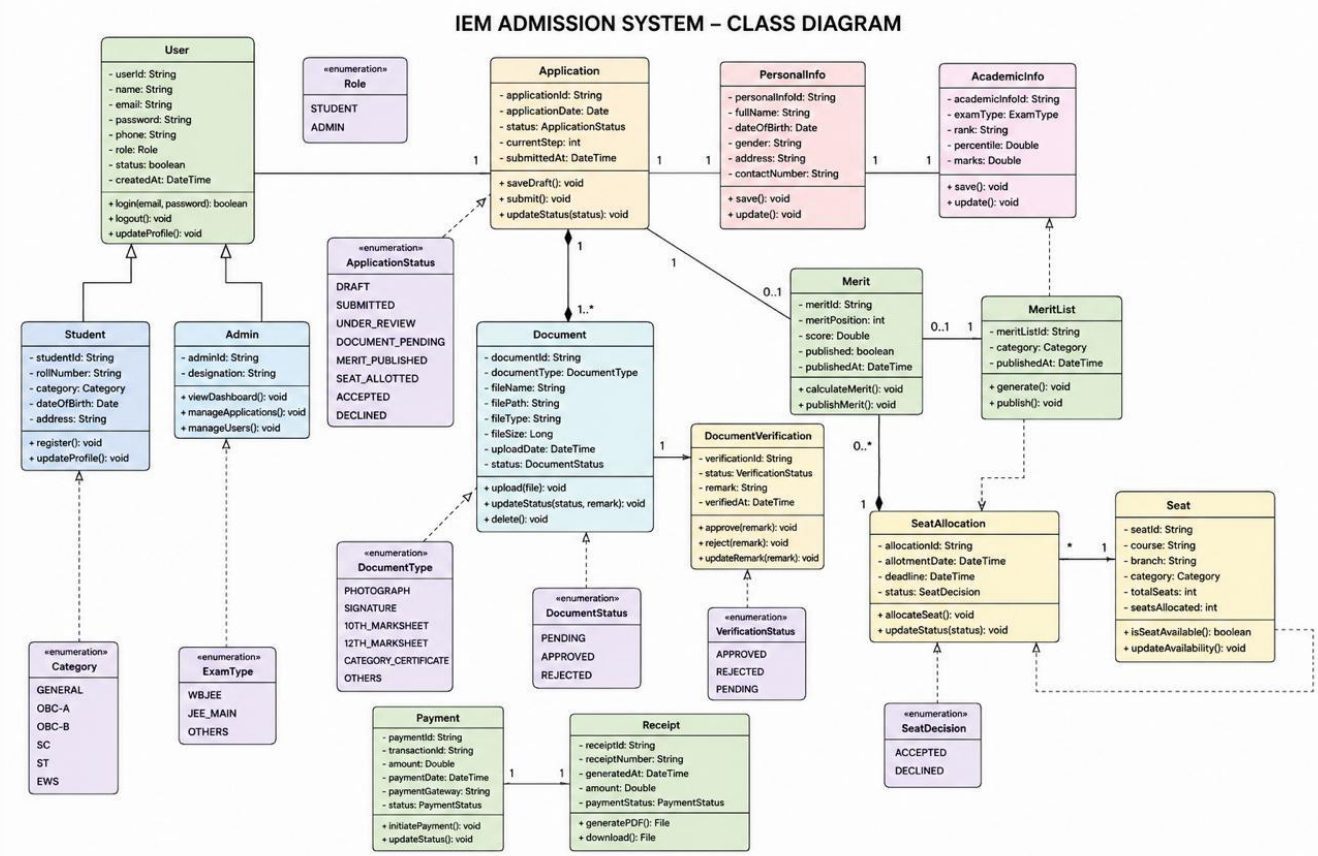
☐ Attributes with data types - Each class has attributes with proper data types like String, DateTime, int, Double, Boolean, and enum types (Role, DocumentType, Category, PaymentStatus)

☐ Methods with parameters and return types - Methods show parameters and return types: login(email, password): boolean, saveDraft(): void, submit(): void, approve(remark): void, calculateMerit(): void, generatePDF(): File

☐ Relationships between classes - User connects to Admin and Application; Application connects to PersonalInfo, AcademicInfo, Document, and Merit; Merit connects to MeritList and Seat; Seat connects to Payment and Receipt; Document connects to DocumentVerification

☐ Multiplicity indicators present - One-to-many relationships shown (User has many Applications, Application has many Documents)

☐ Enumerations properly defined - Enums include Role, DocumentType, DocumentStatus, VerificationStatus, PaymentStatus, Category, and ExamType



5b. HLD / Architecture Diagram

Required Elements Checklist:

☐ Three tiers clearly separated - Presentation, Application, and Data tiers are distinctly labelled and separated

☐ Presentation tier shows UI components - Shows Student/Candidate and Admin interfaces with their respective functions like Register/Login, Upload Documents, Verify Documents, Calculate Merit

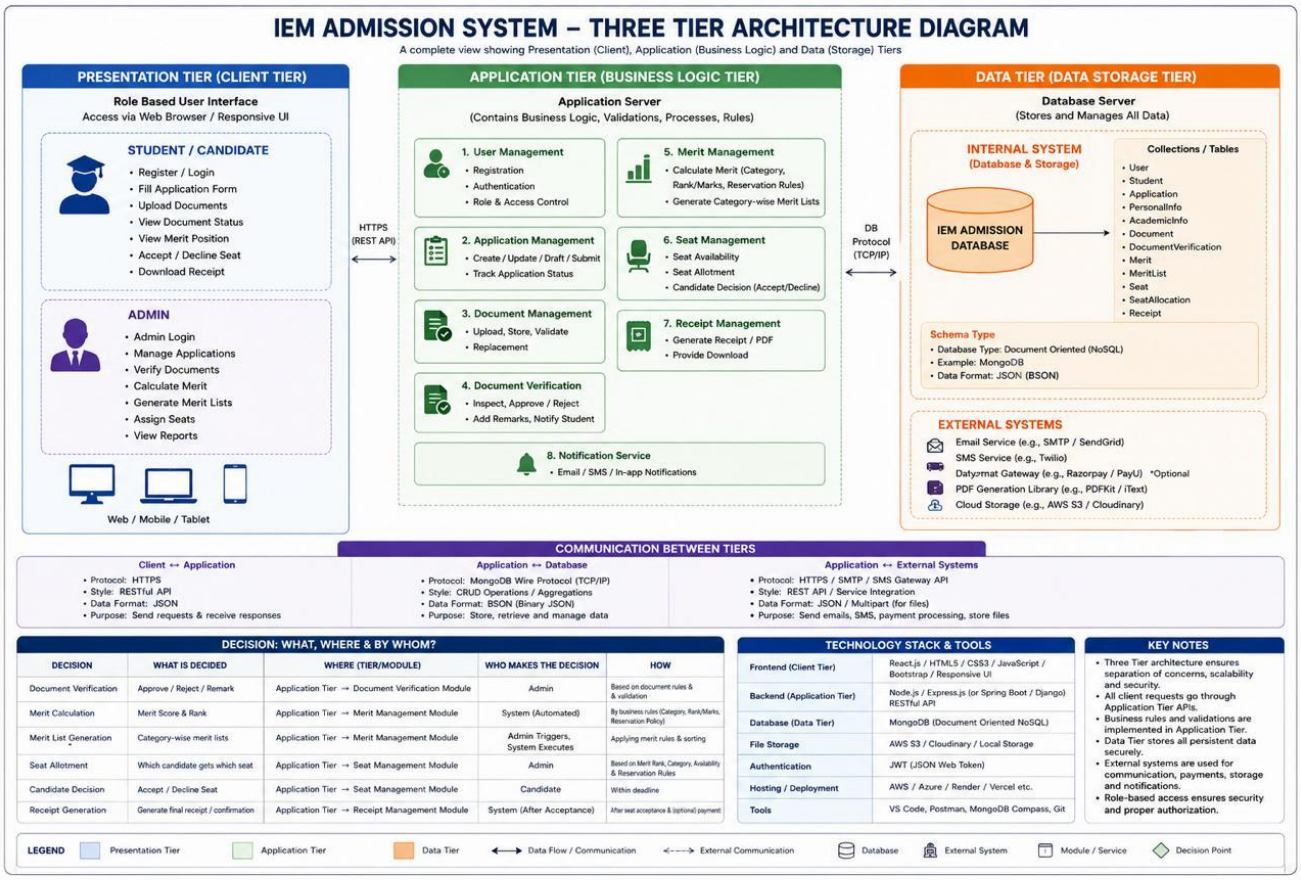
☐ Application tier shows business logic modules - Eight modules listed: User Management, Application Management, Document Management, Document Verification, Merit Management, Seat Management, Receipt Management, and Notification Service

☐ Data tier shows storage components - Lists all database tables: User, Student, Application, PersonalInfo, AcademicInfo, Document, DocumentVerification, Merit, MeritList, Seat, SeatAllocation, Receipt

☐ Communication protocols shown - HTTPS/RESTful API between Client and Application; PostgreSQL protocol between Application and Database; HTTPS/SMTP/SMS Gateway for external systems

☐ Decision table included - Shows WHAT, WHERE, WHO, and HOW for key decisions like Document Verification, Merit Calculation, Seat Allotment, and Receipt Generation

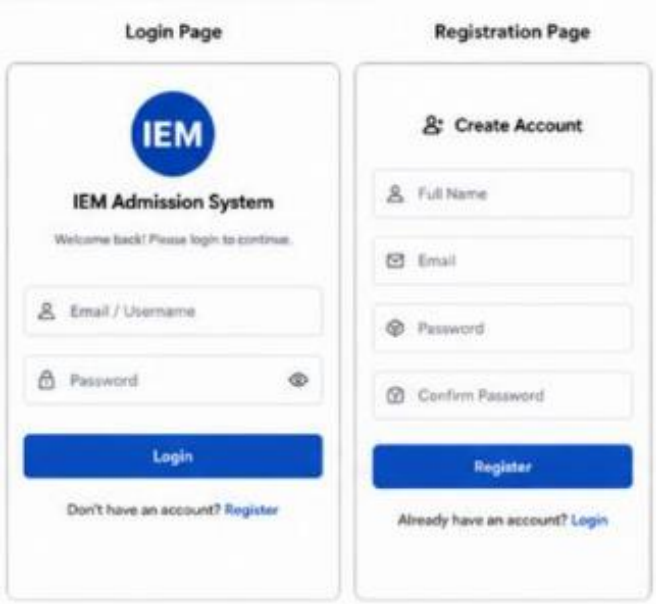
☐ Technology stack specified - Frontend: React.js; Backend: Node.js/Express.js; Database: PostgreSQL; File Storage: AWS S3; Authentication: JWT



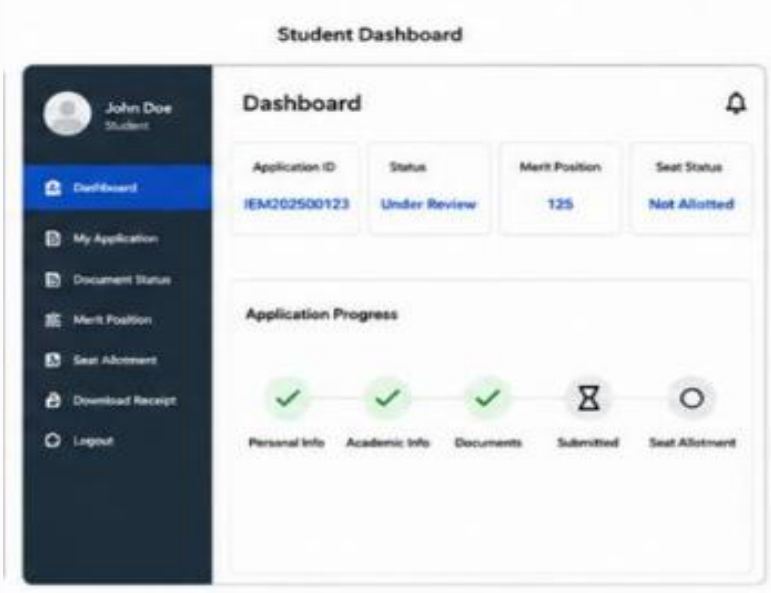
6. UI WIREFRAMES (Figma)

Four core screens, laid out the way they would appear as four linked Figma frames (Login → Dashboard → Document Status → Seat Allotment, with Application Progress as a second view from the Dashboard):

Screen 1 name: Login Page



Screen 2 name: Student Dashboard

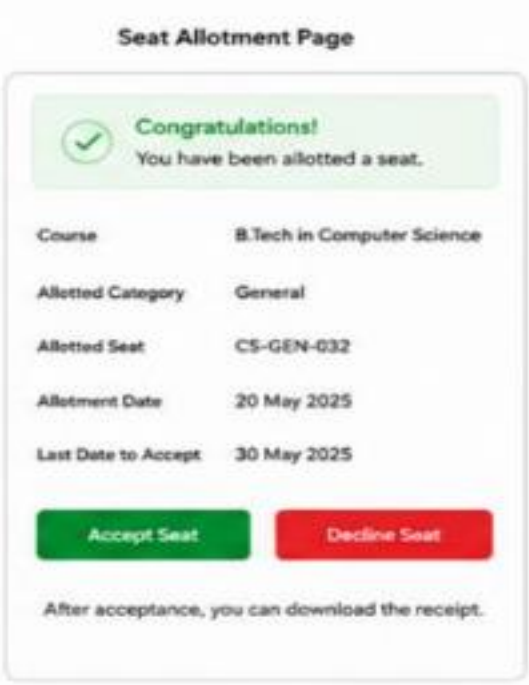


Screen 3 name: Document Status Page



N

Screen 4 name: Seat Allotment Page



Figma "Share prototype" link: _____

7. SELF-ASSESSMENT CHECKLIST

□ Use-case diagram (actors + use cases, correctly notated) - STUDENT and ADMIN actors identified with all use cases properly connected

□ DFD — context diagram AND Level-1 diagram - Context diagram shows CANDIDATE and ADMIN external entities; Level-1 shows six processes with data flows

□ Activity diagram for the chosen use case - Shows complete flow from Register Account to Download Receipt with decision nodes and swimlanes

□ Sequence diagram for the chosen use case - Shows interactions between Student, System, and Admin with numbered messages

□ UML class diagram (or structure chart) as the design diagram - Classes include User, Admin, Application, PersonalInfo, AcademicInfo, Document, Merit, Seat, Receipt, etc.

□ HLD / architecture diagram - Three-tier architecture showing Presentation, Application, and Data tiers with communication protocols

□ 3–4 Figma wireframe screens, grayscale, clearly labelled - Login Page, Student Dashboard, Document Status Page, Seat Allotment Page

□ Figma frames linked into a working clickable flow

□ Figma "Share prototype" link copied above (access set to "Anyone with the link — view")

□ Every diagram uses consistent actor/object names throughout - STUDENT/CANDIDATE and ADMIN used consistently across all diagrams

□ Every team member's name and roll number on the cover page

For Faculty Use

Criterion	Marks Awarded	Remarks
Correct notation (25%)		
Completeness (20%)		
Consistency across diagrams (20%)		
Wireframe clarity & flow (20%)		
Presentation & labelling (15%)		
TOTAL		